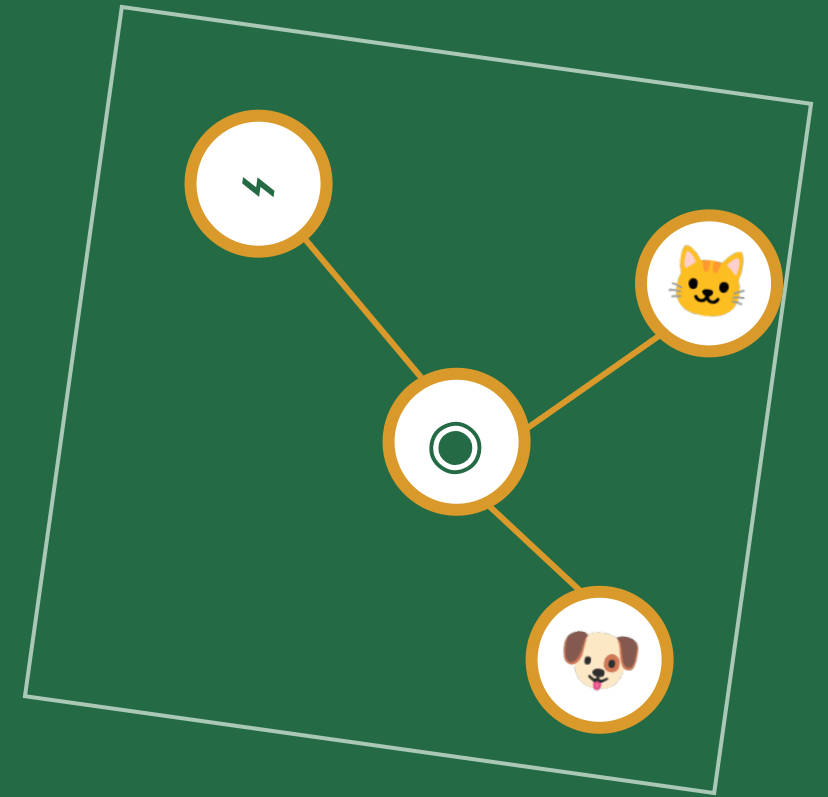


Transfer Learning

¿Por qué entrenar una red neuronal desde cero si otra ya aprendió a ver?



Caso conductor: clasificar gatos 🐱 vs perros 🐶
con MobileNetV2

REUTILIZAR · ADAPTAR · EVALUAR

Pocos datos. Mucho trabajo.

800

Imágenes no son infinitas

Una CNN nueva podría memorizar ejemplos en vez de aprender patrones generales.

Riesgo: overfitting.

GPU?

Entrenar desde cero cuesta

Más épocas, más parámetros y más tiempo antes de obtener una baseline útil.

Pregunta: ¿qué conocimiento podemos reutilizar?

No empezamos por “aprender a ver”. Empezamos por aprovechar lo que ya funciona.

Transferir representaciones visuales

Tarea origen

Una red aprende con un dataset grande, por ejemplo ImageNet.

features aprendidas



bordes · texturas · formas · patrones

Tarea destino

Reutilizamos la base para distinguir gato 🐱 y perro 🐶.



Transfer Learning = reutilizar conocimiento aprendido para resolver un problema nuevo.

Un modelo preentrenado ya trae una memoria visual

MobileNetV2 aprendió patrones generales antes de ver nuestro dataset.



MODELO PREENTRENADO

MobileNetV2

Red entrenada previamente sobre un dataset grande, como **ImageNet**.

`weights = "imagenet"`

Conservamos: la base convolucional y sus representaciones.

Sustituimos: la cabeza original por un clasificador gato/perro.

La red no empieza vacía: empieza sabiendo detectar estructura visual.



Congelar la base. Entrenar la cabeza.

La red ya sabe extraer señales visuales; nosotros cambiamos la decisión final.



QUÉ SE ENTRENA

Base

Congelada: conserva sus pesos preentrenados.

Cabeza

Nueva: aprende desde cero con nuestro dataset.

Ventaja

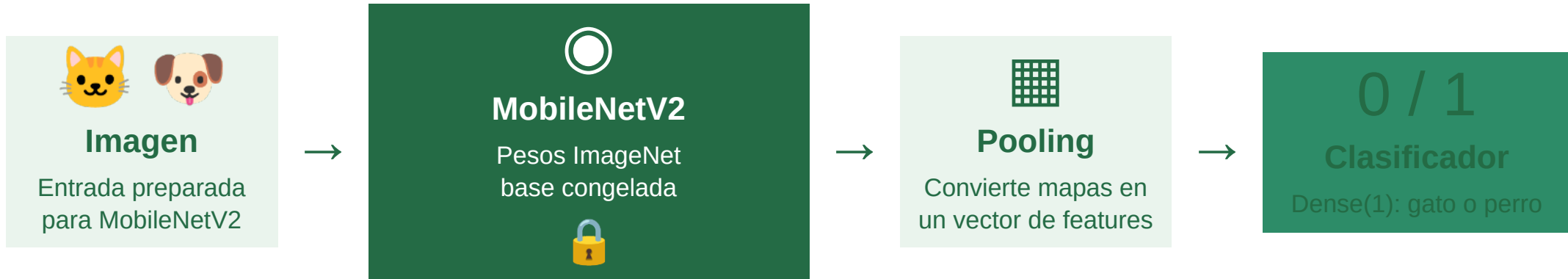
Menos parámetros y entrenamiento más rápido.

Pregunta

¿La baseline ya es suficientemente buena?

Experimento 1: gatos vs. perros

La base ve; la nueva cabeza aprende a decidir.



```
base = MobileNetV2(weights="imagenet", include_top=False)    base.trainable = False
```

Entrenar solo la cabeza nueva · evaluar con el mismo split de validación

¿Qué conseguimos con Feature Extraction?

Una baseline rápida para saber si la representación preentrenada ya es suficiente.

 **BASELINE**

**La base observa.
La cabeza aprende.**

MobileNetV2 permanece congelada; solo se actualiza el clasificador gato/perro.



Accuracy de validación

**[Accuracy
Feature Extraction]**

Parámetros entrenables

**[Trainable
Parameters FE]**

Loss de validación

**[Loss
Feature Extraction]**

Tiempo por época

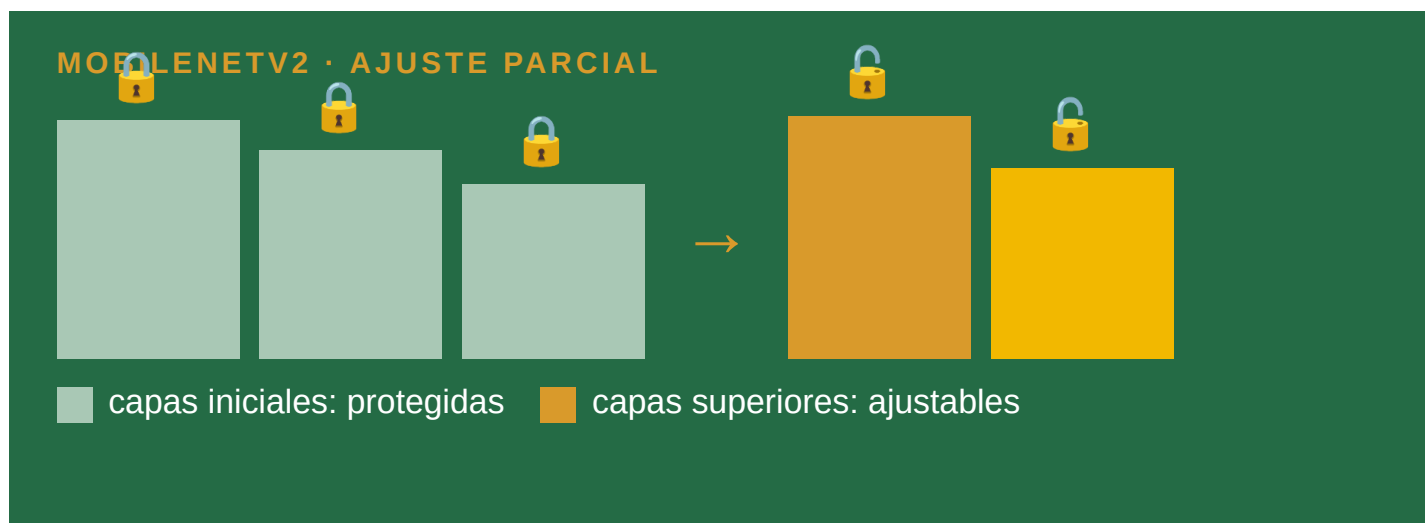
**[Time per
Epoch FE]**

Si la baseline funciona, paramos. Si falta adaptación, desbloqueamos con cuidado.



Descongelar las capas superiores

La cabeza ya aprendió una baseline; ahora adaptamos con cuidado las representaciones más específicas.



NO ES “ENTRENAR TODO”

Adaptar sin borrar lo aprendido

Descongelamos solo parte de la base y entrenamos conjuntamente **cabeza + capas superiores** para que las features profundas sean más relevantes para gatos y perros.

1 · Primero

Entrena la nueva cabeza con la base congelada.

2 · Después

Descongela los últimos bloques de la red.

3 · Siempre

Recompila y valida con actualizaciones pequeñas.

Fine-Tuning = **adaptación incremental**, no reinicio del aprendizaje.

Pequeños pasos. Conocimiento protegido.

En Fine-Tuning no queremos reaprender la visión: queremos reajustarla.

Learning rate grande

Actualizaciones bruscas: la red puede alejarse de una buena representación y sobreajustar rápidamente.



Learning rate baja

Actualizaciones pequeñas: las capas superiores se adaptan de forma incremental al dominio gatos/perros.



Más parámetros

Al descongelar capas, vuelve a entrenarse una red mucho mayor.

Menos datos

Un dataset pequeño aumenta el riesgo de overfitting.

Más control

Pasos pequeños protegen las representaciones preentrenadas.

Ejemplo de configuración: `Adam(learning_rate=1e-5)` · valor orientativo, no universal

Experimento 2: comparar la adaptación

Mismo dataset. Mismo split. Dos estrategias de entrenamiento.



Feature Extraction

Base congelada

solo features

→

Cabeza nueva

se entrena



Fine-Tuning

Últimos bloques

se descongelan

→

LR bajap.ej. 1e-5

MÉTRICAS A COMPLETAR TRAS EJECUTAR AMBOS EXPERIMENTOS

Métrica	 Feature Extraction	 Fine-Tuning
Accuracy de validación	[Accuracy Feature Extraction]	[Accuracy Fine-Tuning]
Loss de validación	[Loss Feature Extraction]	[Loss Fine-Tuning]

Interpretación: comparar la mejora en validación con el coste y el riesgo de **sobreajuste**.

¿Cuál elegirías?

Antes de revelar la recomendación, vota: ¿  Feature Extraction o  Fine-Tuning?

Piensa en datos · similitud de dominio · recursos · comportamiento de la baseline

ESCENARIO A

Pocos datos + dominio parecido

800 imágenes, GPU limitada y fotografías cercanas al tipo de imágenes del preentrenamiento.

¿  o  ?

ESCENARIO B

Más datos + dominio diferente

20 000 imágenes y texturas muy distintas a ImageNet, por ejemplo imágenes médicas.

¿  o  ?

ESCENARIO C

Baseline insuficiente

1 500 imágenes, dominio parecido, pero Feature Extraction no alcanza el objetivo de validación.

¿  o  ?

ORIENTACIÓN DOCENTE

A →  Feature Extraction

B →  Fine-Tuning más profundo

C →  Fine-Tuning parcial + LR baja

No hay una estrategia universal: la decisión depende del tamaño del dataset, la similitud de dominio y la validación.

Tres ideas para recordar

La pregunta inicial cambia de forma: no empezamos desde cero; empezamos desde conocimiento.

01

Reutilizar

Transfer Learning aprovecha representaciones aprendidas en otra tarea visual.



02



Congelar

Feature Extraction mantiene la base fija y entrena un clasificador nuevo.



03



Adaptar

Fine-Tuning descongela parte de la red y ajusta con learning rate baja.

REGLA PRÁCTICA

Empieza con la baseline más segura.
Desbloquea solo si los datos y la validación lo justifican.

Cuando otra red ya aprendió a ver, nuestro trabajo es enseñarle a mirar el nuevo problema **sin borrar lo que sabe.**